THE "MYTHICAL MAN MONTH" AND INTRODUCTION TO SOFTWARE TESTING

CSE 362

Software Development and Professional Practice

Winter 2023

© Royal Military College of Canada

Suppose …

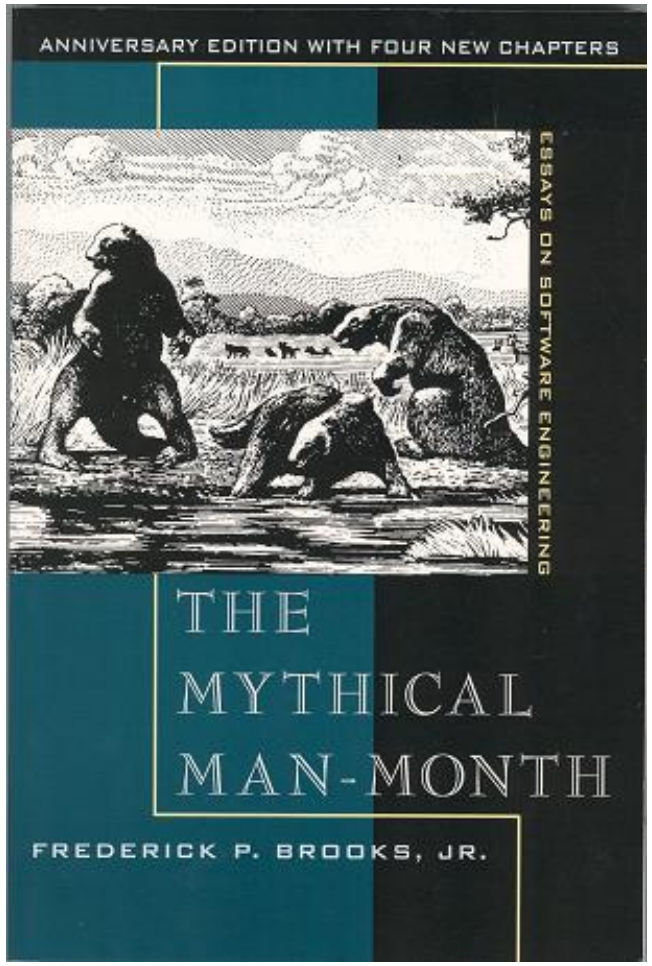
- You have a software project that needs completing.
- Your local software guru says it would take him about 12 months to get it done – 12 "man-months".
- You know your competitor will have a similar product on the shelves in 5 months.
- If you hire another 5 more coders (from "Rent-A-Geek"), can you get the project done in 2 months?

The Issues

- Sub-division of the overall task will take extra time.
- Not all programmers will have equal skill levels – some extra training will be required.
- Working as a team will necessitate extra time for communications!
- But you still have to develop large software projects as a team – the current demand for reduced time to market requires it.

“The Mythical Man-Month”

- by Fred Brooks (Addison-Wesley, 1995)
- Fred was the project manager for IBM System/360 and OS/360.
- The first real treatment of this subject – a classic.



“Brook’s Law”

“Adding manpower to a late software project makes it later.”

- He likens the best software team to a surgical team:
  - One clear leader: the surgeon.
  - With just enough supporting people to make the surgery possible.

- The teams should be as small and “tight” as possible.

Applied to Agile Teams

- Same thing?
- If an agile team is running late on a project, would you add more members to the team and expect to see an improvement?

Applied to Agile Teams, Cont.

Some would say that an XP ("Extreme Programming") team can overcome Brook's Law:

- Integrate new coders by pairing them up with experienced programmers – pair programming.
- The experienced programmers are still producing at the same time as training the newbies.
- Continue to use all the other tools that facilitate rapid development: TDD, Repositories, etc.

Reminder – Extreme Programming

- Design and code at the same time.
- Keep it simple.
- prototyping, refactoring.
- Pair programming.
- Automated unit testing.
- TDD ("Test Driven Development")
- User acceptance testing.

TDD

The three Laws of TDD:

1. You may not write production code until you have written a failing unit test.
2. You may not write more of a unit test than what is sufficient to cause a failure, and not compiling is a failure.
3. You may not write more production code than what is sufficient to pass the current failing test.

TDD, Cont.

- So, tests should be written at the same time as the production code. You are always one test ahead of what the production code can pass.
- Tests are used to express your best understanding of what your code is supposed to do.
- As the amount of production code grows so do the number of tests.
- Testing code should be in separate files from production code.

XP Testing

- Every coder is responsible for building and running their own unit tests on their code.
- Your code goes in the repository after it has passed all of your tests.
- Daily! Before the nightly, automated build.
- Does this mean that the entire system will work as planned?

Two Quotes on Testing

- Kent Beck: "*Optimism is the occupational hazard of programming; testing is the treatment.*"

- Brian Marick: "*The first mistake people make is thinking that the testing team is responsible for assuring quality.*"

Testing

- Testing alone cannot prove conclusively that code is without errors.
- Don't rely on testing to make good code out of bad – write the best code from the best models first!
- Testers should not be beginners! Testing requires knowledge of the entire system and its design, along with knowledge and experience with the testing techniques.

System Failure

- Often the result of a combination of events that is nearly impossible to predict or test for.

- A few examples ranging from just annoying to deadly.

- Classical examples of system failures:
  - Arianne 5 Rocket: overflow problem
  - Patriot Missiles: rounding problem

Phoenix Pay System for the Canadian Government

- IBM was the only company to bid on the creation of a new payroll system in 2011. The contract went to 2019. It is likely worth about $185 million to IBM, due to amendments covering IBM's assistance in fixing the current problems. The original contract was only for $5.7 million.

- IBM was instructed to base the system on PeopleSoft, by Oracle.

- <u>The government was to be responsible for installation, testing and training</u>. They wanted to save money and do it themselves.

Phoenix Pay System, Cont.

- First installed in February 2016.
- There was no proof that the system was working completely, but a $4.8 million award was paid to 340 executives in Public Services and Procurement Canada who installed the system ahead of its scheduled delivery date.
- In April, 2016 the existing 40 year old payroll system was taken off line after the second release of Phoenix was installed with minimal and inadequate testing.

Phoenix Pay System, Cont.

- About eighty thousand government employees have been overpaid or underpaid. And about 700 have not been paid at all!

- This is not the first time that IBM has had problems getting such large systems working. (See the Queensland Health Payroll problems from 2010, for example).

- Problems have been linked to <u>inadequate testing</u>, not enough training and a lack of understanding about how long it would take employees to master the new system.

Phoenix Pay System, Cont.

- The system has also been responsible for 11 widespread internal data breaches of employee data.
- When released, the problems resulted in 7,000 calls per day to a help system that was designed to handle a maximum of 2,200 calls per day.
- In July, 2016, there were 82,000 outstanding employee reported problems with the system.
- It is estimated that by the time another system is installed, it will have cost $2.2 billion to keep Phoenix somewhat running.

Microsoft Zune 30 Meltdown

- Apparently 2006 models of this MP3 player just froze up at midnight, Dec. 31, 2008.

http://slashdot.org/article.pl?sid=09/01/04/2034248

- Eventually Microsoft acknowledged the problem.

- Here is the offending C code:
- Can you spot the problem?

```
#define ORIGINYEAR 1980
BOOL ConvertDays(UINT32 days, SYSTEMTIME* lpTime) { int dayofweek,
  month, year;
  UINT8 *month_tab;
  //Calculate current day of the week
  dayofweek = GetDayOfWeek(days);
  year = ORIGINYEAR;
  while (days > 365) {
    if (IsLeapYear(year)) {
      if (days > 366) {
        days -= 366;
        year += 1;
      }
    }
    else {
      days -= 365;
      year += 1;
    }
  }
…
```

- Intent of the code: repeatedly decrement the days count—by 365 in common years and by 366 in leap years—and each time we do that, we'll increment the year number by 1.
- At the end of the process, the year count should have reached the correct current value.

Next…

Unit testing

Integration and system testing.